

Database Design – TeamMatch

1. Entities

The main entities for TeamMatch are:

- Student
 - Course
 - Instructor
 - Team
 - TeamMember
 - MatchRun
 - StudentProfile
-

2. Attributes

Student

Attribute	Key	Description
student_id	PK	Unique student ID
name		Student name
email		Student email
course_id	FK	Course the student belongs to

Instructor

Attribute	Key	Description
instructor_id	PK	Unique instructor ID
name		Instructor name
email		Instructor email

Course

Attribute	Key	Description
course_id	PK	Unique course ID
course_name		Name of the course
instructor_id	FK	Instructor assigned to course
team_size		Required team size

StudentProfile

Attribute	Key	Description
profile_id	PK	Unique profile ID
student_id	FK	Student linked to profile
skills		Student skills, such as Python or frontend
availability		Student available days/times
preferred_role		Preferred role on a team

Team

Attribute	Key	Description
team_id	PK	Unique team ID
course_id	FK	Course the team belongs to
run_id	FK	Match run that created the team
team_score		Score calculated by matching algorithm

TeamMember

Attribute	Key	Description
team_id	PK, FK	Team ID
student_id	PK, FK	Student ID

MatchRun

Attribute	Key	Description
run_id	PK	Unique match run ID
course_id	FK	Course being matched
created_at		Time the match was generated
status		Status such as completed or failed

3. Primary Keys

- Student.student_id
 - Instructor.instructor_id
 - Course.course_id
 - StudentProfile.profile_id
 - Team.team_id
 - MatchRun.run_id
 - TeamMember.team_id + TeamMember.student_id
-

4. Foreign Keys

- Student.course_id → Course.course_id
- Course.instructor_id → Instructor.instructor_id

- StudentProfile.student_id → Student.student_id
 - Team.course_id → Course.course_id
 - Team.run_id → MatchRun.run_id
 - TeamMember.team_id → Team.team_id
 - TeamMember.student_id → Student.student_id
 - MatchRun.course_id → Course.course_id
-

5. Relationships

- One Instructor can teach many Courses. **1:N**
 - One Course can have many Students. **1:N**
 - One Student has one StudentProfile. **1:1**
 - One Course can have many MatchRuns. **1:N**
 - One MatchRun can create many Teams. **1:N**
 - One Course can have many Teams. **1:N**
 - One Team can have many Students, and one Student can belong to a Team through TeamMember. **M:N resolved using TeamMember**
-

6. ER Diagram

Copy this into your document as the ER diagram text, or recreate it in diagrams.net.

Instructor (1) —————< (N) Course

Course (1) —————< (N) Student

Student (1) ————— (1) StudentProfile

Course (1) —————< (N) MatchRun

MatchRun (1) —————< (N) Team

Course (1) —————< (N) Team

Student (N) —————< TeamMember >————— (N) Team

7. ER Diagram Table View

```
+-----+
| Instructor |
+-----+
| instructor_id PK |
| name          |
| email         |
+-----+
```

|
| 1:N
v

```
+-----+
| Course      |
+-----+
| course_id PK   |
| course_name    |
| instructor_id FK |
| team_size      |
+-----+
```

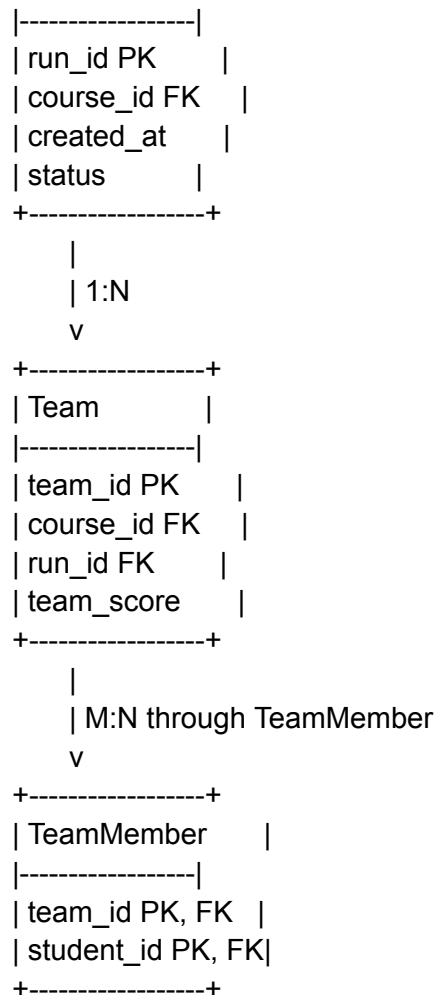
|
| 1:N
v

```
+-----+
| Student      |
+-----+
| student_id PK |
| name          |
| email         |
| course_id FK  |
+-----+
```

|
| 1:1
v

```
+-----+
| StudentProfile |
+-----+
| profile_id PK   |
| student_id FK   |
| skills          |
| availability     |
| preferred_role  |
+-----+
```

```
+-----+
| MatchRun        |
+-----+
```



8. Short Explanation

The TeamMatch database is designed to support student profile collection, course management, and automated team generation. Students belong to courses and submit profile information such as skills, availability, and preferred role. Instructors manage courses and team-size settings. The system uses this stored data to generate teams during a MatchRun.

The TeamMember table is used to handle the many-to-many relationship between students and teams. This avoids duplicating student information inside the Team table and keeps the database organized. Foreign keys connect students, courses, teams, and match runs, which helps maintain data consistency and supports the main TeamMatch features described in the project requirements.